

Konstruct Handover

Konstruct Microservices — Live API & Deployment Guide

Host (SSH): `root@101.53.133.132`

password : RMNPEQ@ndwfg796

Project root: `/opt/projects`

Services present under `/opt/projects` :

```
ChecklistMicroService/  
ConstructnUI/          # React/UI (spelling note)  
OrgnaizationMicroServices/  
ProjectMicroServices/  
UserMicroServices/
```

1) Global Conventions

- **Auth:** JWT via UserMicroServices (SimpleJWT). Access token required in `Authorization: Bearer <token>`.
- **Base URLs** (default Django patterns):
 - **User:** `http://<host>:<port>/api/` (accounts), plus token endpoints.
 - **Organization:** `http://<host>:<port>/api/` (organizations app)
 - **Project:** `http://<host>:<port>/api/` (projects app)
 - **Checklist:** `http://<host>:<port>/api/` (checklists app)
- **Media/Static:** Served via `MEDIA_URL` and `STATIC_URL` (DEBUG may expose via Django). In production, use Nginx.
- **Tenancy:** (If enabled) pass `X-Tenant-Slug` or subdomain; otherwise single-tenant.

Tip: behind Nginx, mount each service on a prefix: /user/, /org/, /proj/, /chk/ to avoid path clashes since all use /api/.

2) User MicroServices (Auth & Tokens)

URLs file (excerpt):

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('api/', include('accounts.urls')),  
    path('api/token/', CustomTokenView.as_view(), name='token_obtain_pair'),  
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),  
    path('api/token/verify/', TokenVerifyView.as_view(), name='token_verify'),  
]
```

2.1 Endpoints

- `POST /api/token/` → obtain tokens
- `POST /api/token/refresh/` → refresh access token
- `POST /api/token/verify/` → verify token
- `GET/POST ... /api/` → user/role management (see `accounts.urls`)

Example

```
curl -X POST http://<host>:<port>/api/token/ \  
-H 'Content-Type: application/json' \  
-d '{"username":"admin","password":"****"}'
```

Returns `{ access, refresh }`.

3) Organization MicroServices

Project root: `/opt/projects/OrganizationMicroServices/`

Site urls:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('organizations.urls')),
]
```

App urls:

```
router = DefaultRouter()
router.register(r'organizations', OrganizationViewSet)
router.register(r'companies', CompanyViewSet)
router.register(r'entities', EntityViewSet)

urlpatterns = [
    path('user-organizationn-info/<int:user_id>/', UserAlloriginazitionINfo.as_view(), name='user-summary'),
    path('All-Info-Through-company-info/<int:company_id>/', CompanyInfoWithEntities.as_view()),
    path('entity-info/<int:user_id>/<int:entity_id>/', EntityInfoWithParents.as_view()),
    path('organizations/by-user/<int:user_id>/', OrganizationByUserView.as_view(), name='organizations-by-user'),
    path('company/get-company-details-by-organization-id/', CompanyDetailsByOrganizationId.as_view(), name='get-company-details-by-organization-id'),
]
urlpatterns += router.urls
```

3.1 Key Routes

- `GET /api/organizations/` CRUD
- `GET /api/companies/` CRUD
- `GET /api/entities/` CRUD
- `GET /api/organizations/by-user/{user_id}/` → orgs for a user
- `GET /api/user-organizationn-info/{user_id}/` → summary

- `GET /api/All-Info-Through-company-info/{company_id}/` → company + entities
- `GET /api/entity-info/{user_id}/{entity_id}/` → entity with parents
- `POST /api/company/get-company-details-by-organization-id/` → details by org id (method per view)

Auth: Bearer token required. Scope: ADMIN/MANAGER typically.

4) Project MicroServices

Project root: `/opt/projects/ProjectMicroServices/projectservice/`

Site urls:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('projects.urls')),
]
```

App urls (excerpt): see `projects/urls.py` .

4.1 Core ViewSets (router)

- `projects` , `purposes` , `phases` , `stages`
- `buildings` , `levels` , `zones` , `subzones` , `rooms`
- `flattypes` , `flats`
- `categories` , `category-level1..6` , and simple variants

4.2 Notable Function Paths

- `GET /api/stages/<stage_id>/next/` → NextStageAPIView
- `GET /api/stages/<stage_id>/info/` → StageInfoAPIView
- `GET /api/phases/by_project/{project_id}/`
- `GET /api/purposes/by_project/{project_id}/`
- `GET /api/get-stage-details-by-project-id/{project_id}/`

- `GET /api/buildings/by_project/{project_id}/`
- `GET /api/levels/by_building/{building_id}/`
- `GET /api/zones/by_building/{building_id}/` and `/by_level/{level_id}/`
- `GET /api/flattypes/by_project/{project_id}/`
- `GET /api/flats/by_building/{building_id}/` , `/by_level/{level_id}/` , `/by_flattype/{flattype_id}/`
- `GET /api/user-stage-role/get-projects-by-user/`
- `GET /api/buildings/with-levels/by_project/{project_id}/` & `with-levels-and-zones`
- `POST /api/buildings/with-levels-zones/bulk-create/`
- `GET /api/flats/by_project/{project_id}/`
- `GET /api/projects/<project_id>/buildings-details/`
- `GET /api/levels-with-flats/{building_id}/`
- `GET /api/projects/by_organization/{organization_id}/`
- `GET /api/projects/by_ownership/`
- `GET /api/org-project-user-summary/`
- `POST /api/units-by-id/`
- Purpose management: `GET/POST /api/all-purposes/` , `POST /api/projects/<id>/activate-purpose/` , `POST /api/client-purpose/` , `GET /api/client-purpose/<client_id>/` , `PATCH /api/client-purpose/<pk>/soft-delete/`
- `GET /api/category-tree-by-project/`
- `GET /api/projects/<project_id>/nested/`

Categories depth: up to **6** levels here (upgrade path to 7 if required).

5) Checklist MicroService (Konstruct Core)

Project root: `/opt/projects/ChecklistMicroService/checklist_service/`

Site urls:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/create/unit-checklist/', CreateChecklistforUnit.as_view(), name='create-checklist-for-unit'),
    path('api/', include('checklists.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

App urls (excerpt):

```
urlpatterns = [
    path('', include(router.urls)),
    path('checklists/stagehistory/summary/', StageHistorySummaryAPIView.as_view()),
    path('checklists/stagehistory/crm-update/', StageHistoryCRMUpdateAPIView.as_view()),
    path('stage-history/', StageHistoryListView.as_view(), name='stage-history-list'),

    path('checklistitem/start/<int:user_id>/<int:item_id>/', StartChecklistItemAPIView.as_view()),
    path('checklistitems/inprogress/<int:user_id>/', ChecklistItemInProgressByUserView.as_view()),
    path('checklistitems/completed/<int:user_id>/', ChecklistItemCompletedByUserView.as_view()),
    path('checklistitems/by-category-status/<int:cat_or_subcat_id>/', ChecklistItemByCategoryStatusView.as_view()),

    path('accessible-checklists/', AccessibleChecklistsAPIView.as_view(), name='accessible-checklists'),
    path('accessible-checklists-unreviewed/', AccessibleChecklistsWithPendingCheckerSubmissionsAPIView.as_view(), name='accessible-checklists-unreviewed'),

    path('flat-report/<int:flat_id>/', FlatReportAPIView.as_view(), name='flat-report'),
```

```
path('create-checklistitemsubmissions-assign/', CreateSubmissionsForChecklistItemsAPIView.as_view(), name='bulk-create-checklistitemsubmissions'),
path('patch-checklistitemsubmission/', PatchChecklistItemSubmissionAPIView.as_view(), name='patch-checklistitemsubmission'),
```

```
path('my-hierarchical-verifications/', MyHierarchicalVerificationsAPIView.as_view(), name='my-hierarchical-verifications'),
path('bulk-verify-submissions/', BulkVerifyChecklistItemSubmissionsAPIView.as_view(), name='bulk-verify-submissions'),
path('verify-checklist-item-submission/', VerifyChecklistItemSubmissionAPIView.as_view(), name='verify-checklist-item-submission'),
```

```
path('checker-verified-inspector-pending/', VerifiedByCheckerPendingInspectorAPIView.as_view()),
path('my-submissions/', MyChecklistItemSubmissions.as_view()),
path('pending-supervisor/', PendingVerificationsForSupervisorAPIView.as_view()),
```

Dashboard analytics

```
path('checklist-analytics/', ChecklistRoleAnalyticsAPIView.as_view(), name='checklist-analytics'),
```

Items by checklist & list filters

```
path('checklist-items/<int:checklist_id>/', ChecklistItemsByChecklistAPIView.as_view(), name='checklist-items-by-checklist'),
path('checklists/filter/', ChecklistByCreatorAndProjectAPIView.as_view(), name='checklist-filter'),
```

Initializer

```
path('start-checklist/<int:checklist_id>/', InitializeChecklistView.as_view(), name='start-checklist'),
path('Checker-New-checklist/', CheckerInProgressAccessibleChecklists.as_view(), name='Checker-View-checklist'),
```

Decision / Transfer rules

```
path('Decision-making-for-Suer-Inspector/', VerifyChecklistItemForChecker
```

```

NSupervisorAPIView.as_view(), name='Decision_api'),
    path('pending-for-maker/', PendingForMakerItemsAPIView.as_view(), name
='pending-for-maker-items'),
    path('mark-as-done-maker/', MArker_DOne_view.as_view(), name='MArker_s
ave'),
    path('Transafer-Rule-getchchklist/', RoleBasedChecklistTRANSFERRULEAPI
View.as_view(), name='MArker_save'),
    path('Supervisor-Pending-work/', PendingForSupervisorItemsAPIView.as_vie
w(), name='pending-supervsiorr-view'),

    # Creation
    path('create/unit-chechklist/', CreateChecklistforUnit.as_view(), name='creat
e-checklist-for-unit'),
    path('user-generated-checklists/', UserGeneratedChecklist.as_view(), name
='user-generated-checklist'),
]

```

5.1 Key Flows

- **Accessible lists:**

- `GET /api/accessible-checklists/` → all you can work on (role filtered)
- `GET /api/accessible-checklists-unreviewed/` → has pending Checker submissions

- **My work queues:**

- `GET /api/checklistitems/inprogress/{user_id}/`
- `GET /api/checklistitems/completed/{user_id}/`
- `GET /api/my-submissions/`

- **Submit / Verify:**

- `POST /api/create-checklistitems submissions-assign/` (bulk create submissions to start work)
- `PATCH /api/patch-checklistitems submission/` (update fields; remarks, status)
- `POST /api/verify-checklist-item-submission/` (Checker/Supervisor decision)

- `POST /api/bulk-verify-submissions/`
- **Maker actions:**
 - `GET /api/pending-for-maker/` → queue
 - `POST /api/mark-as-done-maker/` → submit with media
- **Supervisor queue:**
 - `GET /api/Supervisor-Pending-work/`
- **Inspector pending after Checker:**
 - `GET /api/checker-verified-inspector-pending/`
- **Stage history & CRM/HOTO:**
 - `GET /api/stage-history/`
 - `POST /api/checklists/stagehistory/crm-update/`
 - `GET /api/checklists/stagehistory/summary/`

Reports: `GET /api/checklist-analytics/` , `GET /api/flat-report/{flat_id}/`

Creation: `POST /api/create/unit-checkklist/` , `POST /api/user-generated-checklists/`

Categories: `GET /api/checklistitems/by-category-status/{cat_or_subcat_id}/`

5.2 Request Examples

1) Get accessible checklists (with Bearer token)

```
curl -H "Authorization: Bearer $TOKEN" \
  http://<host>:<chk_port>/api/accessible-checklists/
```

2) Start a checklist explicitly

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
  http://<host>:<chk_port>/api/start-checklist/123/
```

3) Maker marks done

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
  -F "submission_id=555" -F "maker_remarks=Done" -F "maker_media=@/path/photo.jpg" \
```

```
http://<host>:<chk_port>/api/mark-as-done-maker/
```

4) Checker decision

```
curl -X POST -H "Authorization: Bearer $TOKEN" \  
-H 'Content-Type: application/json' \  
-d '{"submission_id":555, "decision":"pass", "remarks":"ok"}' \  
http://<host>:<chk_port>/api/verify-checklist-item-submission/
```

5.3 Flags & Rules

- **Skip Supervisor:** expose `skip_supervisory=true` on checklist/stage config → Maker submits back to Checker.
- **Repeat Checklist:** `repeat_checklist=true` → on Checker final pass, clone to next stage and assign to next Checker.
- **Multi-image evidence:** multipart arrays or separate `/images/` attach API (if exposed). Legacy single-image fields still present.

6) Process & State (Recap)

- **Normal:** Checker pass → Completed; fail → Maker → Supervisor → Checker (loop).
- **Skip-Supervisor:** Checker fail → Maker → Checker.
- **Inspector (optional):** After Checker pass, pending Inspector audits (if enabled).
- Stage progression recorded in `StageHistory` with CRM/HOTO markers.

7) Server Layout & Run Commands

Directory checks

```
ssh root@101.53.133.132  
cd /opt/projects  
ls -la
```

Project service files (`/opt/projects/ProjectMicroServices/projectservice/projectservice/`):

```
__init__.py asgi.py settings.py urls.py wsgi.py (__pycache__/ present)
```

Run (dev)

```
cd /opt/projects/ProjectMicroServices/projectservice
source venv/bin/activate # if present
python manage.py runserver 0.0.0.0:8002
```

Do similarly for **Organization** (e.g., `:8001`), **User** (`:8000`), **Checklist** (`:8003`).

PM2 + Gunicorn (prod sketch)

```
pm2 start gunicorn --name chk \
  -- gunicorn checklist_service.wsgi:application -b 0.0.0.0:8003 -w 3
pm2 save
```

Nginx mapping (example)

```
/location /user/ { proxy_pass http://127.0.0.1:8000; }
/location /org/ { proxy_pass http://127.0.0.1:8001; }
/location /proj/ { proxy_pass http://127.0.0.1:8002; }
/location /chk/ { proxy_pass http://127.0.0.1:8003; }
```

8) Environment & Settings

- Ensure `.env` has DB creds, SECRET_KEY, DEBUG flag, MEDIA paths, CORS.
- If using SQLite in project service (present `db.sqlite3`), plan migration to Postgres.
- Configure `ALLOWED_HOSTS=["101.53.133.132", "*"]` for dev; restrict in prod.

9) Troubleshooting

- **401 Unauthorized:** missing/expired token → refresh or re-login.

- **404:** wrong path or trailing slash; check service base (`/api/`).
 - **CORS:** update `CORS_ALLOWED_ORIGINS` and `CSRF_TRUSTED_ORIGINS` .
 - **Media upload fails:** ensure `MEDIA_ROOT` writable and Nginx location for `/media/` .
 - **Router basename errors:** set `basename=` for ViewSets lacking `queryset` (done for `transfer-rules` , `rooms`).
-

10) Postman/Collections

- Create one environment per service with `base_url` and `token` .
 - Folders: **User, Org, Project, Checklist.**
 - Save sample bodies for Maker submit, Checker verify, Stage next, CRM update.
-

11) Next Steps

1. Stabilize base prefixes behind Nginx (`/user` , `/org` , `/proj` , `/chk`).
 2. Promote Project service categories to **7 levels** if Konstruct requires it end-to-end.
 3. Add OpenAPI (drf-spectacular) for each service and publish at `/schema/` + `/docs/` .
 4. Switch SQLite → Postgres; add backups.
 5. Wire role flags: `skip_supervisory` , `repeat_checklist` in Checklist create/update APIs.
-

12) Quick Spec Block — Flows & Flags (copy-paste)

Single flow (normal = Supervisor compulsory)

```
Checker →  
  YES → PASS (complete)  
  NO  → Maker → Supervisor → Checker (loop until PASS)
```

Skip Supervisor path

Flag: skip_supervisory = true

Checker (NO) → Maker → Checker (loop until PASS)

Auto-advance to next stage's Checker

After Checker PASS → go direct to Stage N+1 Checker

Repeat checklist across stages

Flag: repeat_checklist = true

On Checker PASS → clone same checklist → assign to next checklist/stage

Multi-image support

✓ Supported at every role step via ChecklistItemSubmissionImage (role-tagged)

Multi-category (7 layers)

category + category_level1 ... category_level7

Stage-specific checklists

Checklist of Stage 1 and Stage 2 can be different (separate templates)

Very clear ownership for DONE

MAKER marks work done (submit with evidence)

SUPERVISOR approves/rejects (normal flow)

CHECKER gives final PASS/FAIL (authoritative) and can advance stage

Transfer rule — main entry

Endpoint (Checklist Service): /api/Transfer-Rule-getchcklist/

View: RoleBasedChecklistTRANSFERRULEAPIView ← main routing/decision A

13) Microservices (4 services)

1. **UserService** — users, access, roles (RBAC)
2. **OrganizationService** — Org, Company, Entity
3. **ProjectService** — Project, Purpose, Phase, Stage; Tower/Building, Level (floor), Zone, Subzone, Flat Type, Flat (unit), transfer rules
4. **Checklist (Konstruct)** — checklists, items, options, submissions, stage history

Location stack (ProjectService terms):

Project → Tower/Building → Level (Floor) → Zone → Subzone → Flat Type → Flat (Unit) → (Checklist Level)